DOCUMENTAÇÃO DE ENTREGA · MVP

Modelo de classes

App de Organização Familiar
Diagramas por domínio, contratos e rastreabilidade

O App de Organização Familiar é um MVP para distribuir responsabilidades domésticas, acompanhar tarefas e envolver adultos e crianças em missões e conquistas coletivas. Administrador, Membro e Kids são os perfis de acesso; o Criador da Família é um Administrador com proteções adicionais.

Modelo 1.0 · Base funcional 1.4 · Referência: 05/10/2026
Edição de entrega: 05 de outubro de 2026

GUIA DE LEITURA

Objetivo e escopo

Diagramas por domínio, contratos e rastreabilidade. O App de Organização Familiar é um MVP para distribuir responsabilidades domésticas, acompanhar tarefas e envolver adultos e crianças em missões e conquistas coletivas. Administrador, Membro e Kids são os perfis de acesso; o Criador da Família é um Administrador com proteções adicionais.

Conteúdo deste documento

1. Rastreabilidade por caso de uso e requisitos de qualidade
2. Seis diagramas de domínio e catálogo de tipos
3. Relacionamentos, regras centrais e correspondência com o modelo de dados
4. Definições pendentes

Convenções e referências

RN: regra de negócio · RF: requisito funcional · RNF: requisito não funcional · UC: caso de uso · TEL: tela · MOD: modal ou painel. Os códigos preservam a rastreabilidade entre os documentos da entrega.

“Adulto” designa Administrador e Membro. Kids possuem permissões próprias. As referências por código permitem consultar os demais volumes sem depender de arquivos de edição ou de navegação em um repositório.

Os contratos usam notação UML e tipos Swift. São especificações de modelagem, não código executável. O catálogo acompanha os diagramas e detalha atributos e operações.

> Documento complementar: **Visão, regras e especificação de requisitos**, que reúne as definições RN, RF, RNF e os fluxos UC01 a UC19. Os diagramas registram modelagem e comportamento esperado; não comprovam implementação concluída.

Como ler

- Os diagramas mostram os tipos e relacionamentos de cada domínio. O catálogo detalha atributos e operações; os identificadores em inglês são preservados para uso em Swift.
- class, struct, enum e protocol indicam o tipo do modelo. Nos diagramas, classes usam a declaração padrão, sem estereótipo; os demais tipos são identificados por estereótipos. + significa público e -, privado. Os blocos são pseudocódigo com notação UML e tipos Swift, não código compilável.
- *-- representa composição; o--, agregação; -->, associação; ..>, dependência. Multiplicidades nas pontas indicam quantas instâncias podem participar; dependências não têm multiplicidade.
- Atributos públicos representam leitura do estado; alterações passam pelas operações autorizadas, sem escrita direta que contorne as regras. As assinaturas representam contratos, não uma escolha de banco ou framework.
- Composição expressa pertencimento no modelo, sem autorizar exclusão em cascata de histórico. Enums são tipos de valor, sem cardinalidade de entidades persistidas.
- A separação dos domínios segue RNF14. PersonalTasks e FamilyArea são coordenadores de apresentação; serviços orquestram operações e protocolos delimitam integrações.

Rastreabilidade por caso de uso

Os RF e RN são os vínculos declarados em cada UC na base funcional. A lista de tipos indica participação no fluxo como um todo; não atribui todos os requisitos a cada classe isoladamente. AccessControl, Action, DomainError e a validação de sessão apoiam transversalmente as operações protegidas. Os aspectos de interface permanecem no screen map e nos user flows.

UC01 - Gerenciar família

Tipos responsáveis ou de apoio: Family, Member, AccessService, AppleAuthenticating

Requisitos funcionais: RF01, RF38, RF39

Regras de negócio: RN01, RN09, RN44

UC02 - Gerenciar membros

Tipos responsáveis ou de apoio: Family, Member, MemberRole, Invite, InviteStatus, Session, AccessService, AccessControl, MemberService, InviteCredentials

Requisitos funcionais: RF02, RF03, RF04, RF05, RF06, RF40, RF41, RF54

Regras de negócio: RN01, RN02, RN03, RN04, RN05, RN06, RN07, RN08, RN10, RN45, RN46, RN47

UC03 - Validar conclusão de tarefa Kids

Tipos responsáveis ou de apoio: FamilyTask, KidsSignal, ValidationDecision, Completion, TaskService, MissionService

Requisitos funcionais: RF25, RF26, RF55, RF56

Regras de negócio: RN22, RN23, RN24, RN25, RN26, RN30, RN35, RN54, RN55, RN56

UC04 - Autenticar com Apple

Tipos responsáveis ou de apoio: AppleAuthenticating, AccessService, Family, Member

Requisitos funcionais: RF37, RF39

Regras de negócio: RN44

UC05 - Gerenciar missões

Tipos responsáveis ou de apoio: Mission, MissionStatus, FamilyTask, Badge, BadgeAward, MissionService

Requisitos funcionais: RF28, RF29, RF31, RF32, RF50, RF58

Regras de negócio: RN16, RN35, RN36, RN37, RN38, RN39, RN40, RN50, RN51, RN53, RN58, RN59, RN64

UC06 - Gerenciar itens de recompensas coletivas

Tipos responsáveis ou de apoio: RewardItem, RewardItemSource

Requisitos funcionais: RF33, RF34

Regras de negócio: RN41, RN42

UC07 - Gerenciar tarefas

Tipos responsáveis ou de apoio: FamilyTask, EffortWeight, TaskStatus, TaskService, ChangeScope, Routine

Requisitos funcionais: RF07, RF08, RF09, RF10, RF15, RF16, RF27, RF58, RF59

Regras de negócio: RN11, RN12, RN13, RN17, RN18, RN19, RN20, RN30, RN38, RN50, RN53, RN57, RN58, RN59

UC08 - Definir recorrência de rotina

Tipos responsáveis ou de apoio: Routine, RoutineFrequency, FamilyTask, TaskService, ChangeScope

Requisitos funcionais: RF13, RF14

Regras de negócio: RN11, RN14, RN16, RN17

UC09 - Ingressar por convite

Tipos responsáveis ou de apoio: Invite, InviteStatus, Session, AccessService, InviteCredentials

Requisitos funcionais: RF42, RF43, RF44, RF45, RF46, RF54

Regras de negócio: RN10, RN45, RN46, RN47

UC10 - Editar perfil

Tipos responsáveis ou de apoio: Member, BadgeAward, Badge, MemberService, MissionService

Requisitos funcionais: RF48, RF62, RF65

Regras de negócio: RN63, RN64, RN33

UC11 - Visualizar minhas tarefas

Tipos responsáveis ou de apoio: FamilyArea, FamilyTask, CarouselRange, ViewPeriod, PeriodProgress, Member, PersonalTasks, WeeklyRecognition, PeriodCalculator

Requisitos funcionais: RF60, RF11, RF17, RF22, RF49, RF65, RF66

Regras de negócio: RN60, RN16, RN22, RN26, RN27, RN33, RN49, RN55, RN28, RN61, RN63

UC12 - Concluir tarefa

Tipos responsáveis ou de apoio: FamilyTask, Completion, CompletionSource, TaskService, MissionService

Requisitos funcionais: RF12, RF47, RF56

Regras de negócio: RN21, RN22, RN30, RN35, RN48, RN55, RN56

UC13 - Visualizar carga diária ou semanal

Tipos responsáveis ou de apoio: FamilyArea, PeriodCalculator, PeriodLoad, Completion, ViewPeriod

Requisitos funcionais: RF18, RF19, RF20, RF21, RF56, RF57, RF66

Regras de negócio: RN26, RN28, RN30, RN31, RN33, RN34, RN35, RN55, RN56, RN57, RN58, RN60, RN61

UC14 - Acompanhar missões e recompensas

Tipos responsáveis ou de apoio: Mission, MissionStatus, BadgeAward, Badge, RedeemedReward, MissionService

Requisitos funcionais: RF30, RF31, RF36, RF58, RF62, RF65

Regras de negócio: RN26, RN35, RN36, RN38, RN39, RN43, RN51, RN52, RN53, RN55, RN58, RN33, RN63, RN64

UC15 - Sinalizar tarefa como realizada

Tipos responsáveis ou de apoio: FamilyTask, KidsSignal, TaskStatus, TaskService, PersonalTasks

Requisitos funcionais: RF23, RF24, RF55, RF56

Regras de negócio: RN21, RN22, RN23, RN24, RN30, RN35, RN54, RN55, RN56

UC16 - Resgatar recompensa da missão

Tipos responsáveis ou de apoio: RewardWheel, RewardItem, RedeemedReward, Mission, MissionService

Requisitos funcionais: RF35, RF51, RF52, RF53

Regras de negócio: RN36, RN38, RN42, RN43, RN52, RN53

UC17 - Retornar tarefa concluída para Pendente

Tipos responsáveis ou de apoio: FamilyTask, Completion, TaskService, Mission, MissionService

Requisitos funcionais: RF57, RF58, RF59

Regras de negócio: RN38, RN50, RN53, RN55, RN57, RN58, RN59

UC18 - Gerar relatório semanal em PDF

Tipos responsáveis ou de apoio: WeeklyReport, WeeklyReportService, MemberReportLine, PDFRendering, FamilyArea, PeriodCalculator, PeriodLoad, PeriodProgress

Requisitos funcionais: RF61

Regras de negócio: RN62

UC19 - Apurar e compartilhar destaque semanal

Tipos responsáveis ou de apoio: WeeklyHighlight, WeeklyHighlightService, HighlightNotifying, PeriodLoad, WeeklyRecognition, WeeklyHighlightScheduling, HighlightSharing, PeriodCalculator

Requisitos funcionais: RF63, RF64

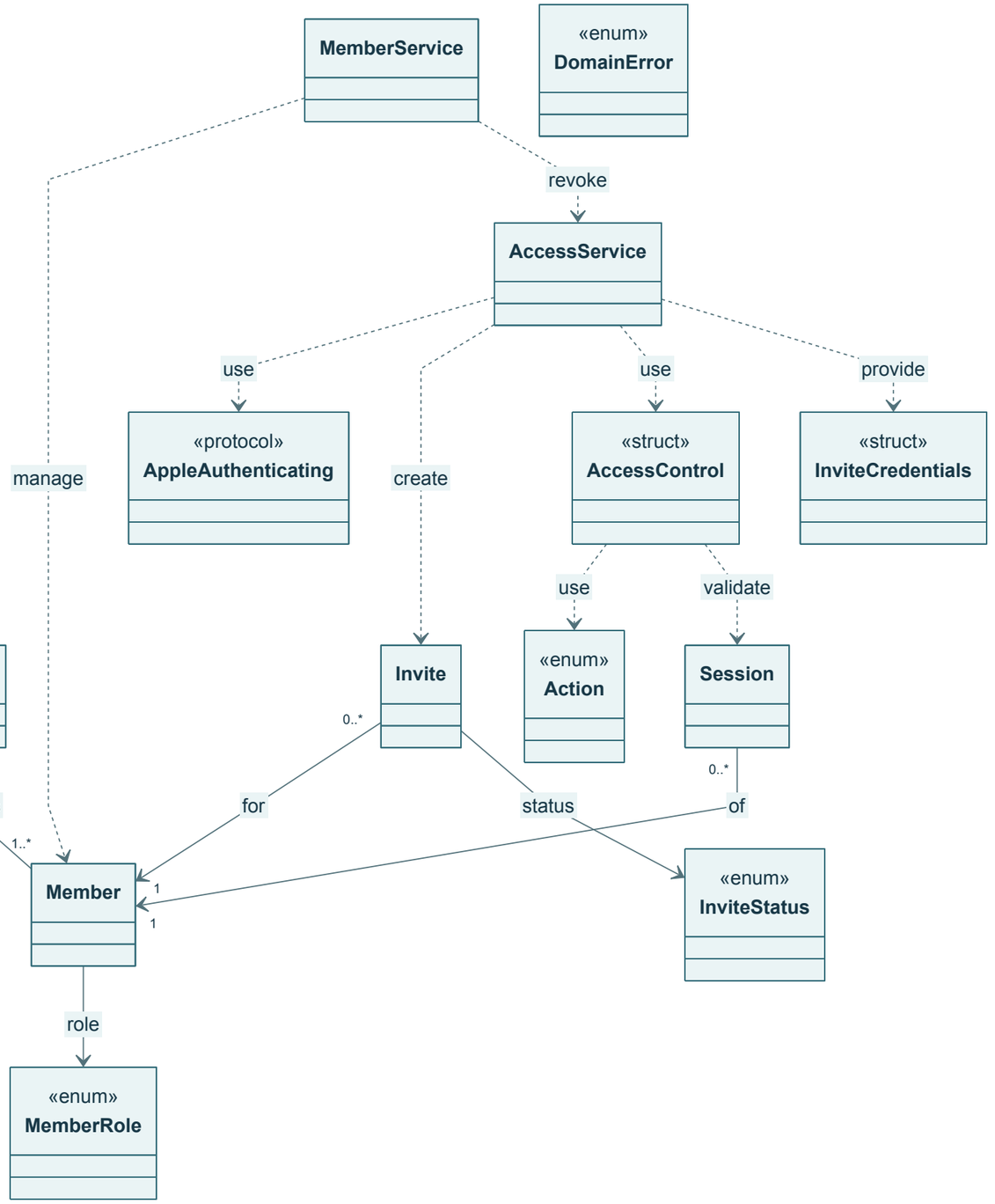
Regras de negócio: RN28, RN30, RN65

Restrições não funcionais

Os contratos de serviço incluem autorização e persistência consistente no backend. A lista completa de RNF aplicáveis permanece em cada UC.

Responsabilidade	Tipos envolvidos	Referências e obrigação
Isolamento e autorização	AccessControl, Session, serviços	RNF06, RNF07 e RNF22: validar família, perfil ativo e permissão no backend em cada operação protegida.
Apple e convites	AppleAuthenticating, AccessService, Invite	RNF15, RNF16, RNF17, RNF18, RNF19, RNF20 e RNF21: integração oficial, proteção de credenciais, validação de vínculo, contenção de tentativas e consumo único atômico.
Conclusão, reabertura e crédito	FamilyTask, Completion, TaskService, PeriodCalculator, Mission	RNF09, RNF10, RNF11 e RNF25: preservar histórico e processar decisões concorrentes sem duplicar crédito ou modificar conquistas.
Resgate	RewardWheel, Mission, RedeemedReward	RNF23: persistir recompensa e encerramento do resgate em uma operação consistente.
Fotos e perfil	Member, KidsSignal	RNF24 e RNF27: controlar acesso às fotos e edição de apelido por titularidade.
PDF e apuração	WeeklyReportService, PDFRendering, WeeklyHighlightService, HighlightNotifying	RNF26 e RNF27: leitura consistente, processamento recuperável, notificações sem duplicidade e compartilhamento explícito.
Apresentação e infraestrutura	Coordenadores, serviços e adaptadores futuros	RNF01, RNF02, RNF03, RNF04, RNF05, RNF08, RNF12 e RNF13: usabilidade, acessibilidade, desempenho, erros de persistência e responsividade dependem da implementação e de validação além deste modelo.

Família, perfis e acesso



App de Organização Familiar · Base funcional 1.4 · Figura 2

O recorte mostra as relações internas deste domínio; as relações entre domínios estão na tabela de relacionamentos.

Family «class»

Rastreabilidade: UC01, UC02, UC04.

```
+ id: UUID
+ name: String
+ createdAt: Date
+ members: [Member]
+ administrators: [Member] { get }
+ creator: Member { get }
+ carouselRange(today: Date, calendar: Calendar) -> CarouselRange
```

Restrições: Um único Criador, sempre Administrador; a família tem ao menos esse perfil. O retorno autenticado à família existente não cria nova família (RN01, RN05, RN09).

Member «class»

Rastreabilidade: UC01, UC02, UC04, UC10, UC11.

```
+ id: UUID
+ familyID: UUID
+ name: String
+ nickname: String?
+ photoURL: URL?
+ role: MemberRole
+ isFamilyCreator: Bool
- appleUserID: String?  // somente o Criador
+ removedAt: Date?
+ isActive: Bool { get }  // removedAt == nil
+ isAdministrator: Bool { get }
+ isAdult: Bool { get }
+ isKids: Bool { get }
+ editAdministrativeName(_ name: String, by admin: Member) throws
+ editOwnProfile(nickname: String?, photoURL: URL?, by holder: Member) throws
+ promote(by admin: Member) throws
+ demote(by admin: Member) throws
+ leaveAdministration(by holder: Member) throws
+ setKidsMode(_ enabled: Bool, by admin: Member) throws
```

Restrições: Nome de identificação e apelido são campos separados. Gestão administrativa não altera foto/apelido pessoais; edição pessoal não muda família, papel ou modo Kids. Apelido é visível à família, mas sua configuração é exclusiva do titular. Promoção exige adulto; Criador não pode ser rebaixado, removido ou deixar a administração; Administrador não pode ser Kids (RN02, RN03, RN04, RN05, RN06, RN07, RN08, RN44, RN63).

MemberRole: String, Codable «enum»

Rastreabilidade: UC02.

```
case administrator
case member
case kids
```

Invite «class»

Rastreabilidade: UC02, UC09.

```
+ id: UUID
+ familyID: UUID
+ memberID: UUID
- tokenVerifier: Data
- passwordVerifier: Data
+ createdBy: UUID
+ createdAt: Date
+ consumedAt: Date?
+ revokedAt: Date?
+ status: InviteStatus { get }
+ validate(token: String, password: String, member: Member) throws
```

Restrições: Vinculado à família e ao perfil ativo; link e senha admitem um único ingresso. Validação sem sucesso não consome o convite. Não há expiração temporal especificada. A proteção das credenciais não permite senha legível persistida; sua disponibilização para reenvio é responsabilidade do serviço de acesso, sem impor rotação de link/senha (RN45, RN46, RN47, RNF17, RNF18, RNF19, RNF20, RNF21).

InviteStatus: String, Codable «enum»

Rastreabilidade: UC02, UC09.

```
case available
case consumed
case invalidated
```

Session «class»

Rastreabilidade: UC02, UC09.

```
+ id: UUID
+ memberID: UUID
+ familyID: UUID
- refreshTokenVerifier: Data
+ createdAt: Date
+ revokedAt: Date?
+ validate(member: Member, familyID: UUID) throws
+ refresh(refreshToken: String, member: Member) throws -> Session
+ revoke()
```

Restrições: Validação e renovação conferem revogação, perfil ativo e vínculo à família. Refresh token não expira por tempo. Remoção revoga sessões existentes e renovações (RN10, RN47, RNF22).

AppleAuthenticating «protocol»

Rastreabilidade: UC01, UC04.

```
+ authenticate() async throws -> String  // appleUserID
```

AccessService «class»

Rastreabilidade: UC01, UC02, UC04, UC09.

```
- apple: AppleAuthenticating
+ signInWithApple() async throws -> Family?
+ createFamily(name: String) async throws -> Family
+ generateInvite(for memberID: UUID, by admin: Member) async throws
      -> InviteCredentials
+ resendInvite(_ inviteID: UUID, by admin: Member) async throws
      -> InviteCredentials
+ join(token: String, password: String) async throws -> Session
+ refreshSession(_ sessionID: UUID, refreshToken: String) async throws -> Session
+ revokeAccess(of memberID: UUID, by admin: Member) async throws
```

Restrições: Apple autentica exclusivamente o Criador e recupera sua família existente. Gerar convite usa perfil já criado; reenviar apenas disponibiliza o convite ainda disponível para envio conjunto de link e senha, sem mudar seu estado ou presumir rotação. Convite consumido ou recuperação exigem novo convite para o mesmo perfil ativo. Ingresso valida e consome o conjunto junto à criação da sessão em operação atômica, com proteção contra tentativas abusivas. Credenciais transitórias não são registradas em logs (UC01, UC02, UC04, UC09, RNF15, RNF16, RNF17, RNF18, RNF19, RNF20, RNF21, RNF22).

InviteCredentials «struct»

Rastreabilidade: UC02, UC09.

```
+ inviteID: UUID
+ link: URL
+ password: String
```

Restrições: Dados transitórios para disponibilizar link e senha juntos. Não são a representação persistida do convite (RN45, RNF17).

MemberService «class»

Rastreabilidade: UC02, UC10.

```
- access: AccessService
+ addMember(name: String, role: MemberRole, by admin: Member) async throws -> Member
+ editMember(_ memberID: UUID, name: String, by admin: Member) async throws
+ removeMember(_ memberID: UUID, by admin: Member) async throws
+ editOwnProfile(nickname: String?, photoURL: URL?, by holder: Member) async throws
```

Restrições: Gestão valida mesma família e privilégio administrativo. Remoção aplica de forma consistente suas alterações e protege o Criador, revoga acesso inclusive a fotos/sessões, deixa as tarefas futuras afetadas sem responsável e preserva o histórico concluído. Alterar foto/apelido só é efetivado após sucesso da persistência; falha mantém os dados anteriores (RN07, RN10, RN63, RNF12, RNF24, RNF27).

AccessControl «struct»

Rastreabilidade: Todos os casos de uso.

```
+ static require(_ action: Action, session: Session, actor: Member,
    resourceFamilyID: UUID, ownerID: UUID?) throws
+ static can(_ action: Action, session: Session, actor: Member,
    resourceFamilyID: UUID, ownerID: UUID?) -> Bool
```

Restrições: O backend valida sessão, perfil ativo, família do recurso, papel e titularidade quando aplicável. ownerID identifica o titular do perfil ou reconhecimento; sinalização Kids também valida responsabilidade na tarefa. Ocultar ação na interface não substitui autorização (RNF06, RNF07, RNF22, RNF27).

Action «enum»

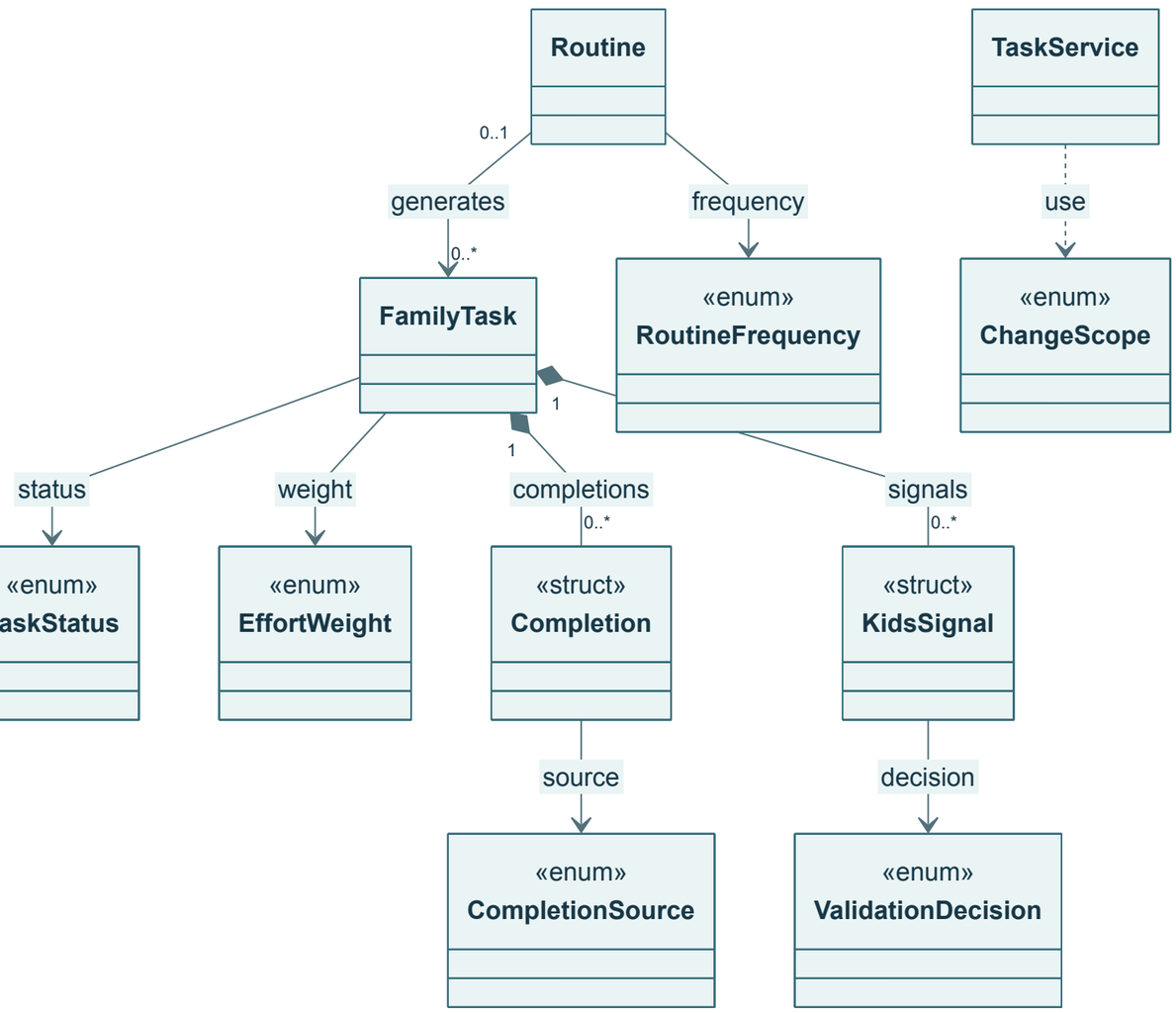
Rastreabilidade: Todos os casos de uso.

```
case manageMembers, generateInvite, resendInvite
case manageTasks, completeTask, signalTask, validateSignal, reopenTask
case manageMissions, manageRewardItems, redeemReward
case viewOwnTasks, viewOwnProgress, viewMissions
case viewLoad, accessFamilyArea, generateWeeklyReport
case viewOwnWeeklyHighlight, shareOwnWeeklyHighlight
case editOwnProfile
```

DomainError: Error «enum»

Rastreabilidade: Todos os casos de uso.

```
case permissionDenied
case creatorProtected
case administratorInKidsMode
case titleRequired, invalidWeight
case completedTaskNotDeletable
case completedMissionProtected
case missionWithoutTasks
case invalidInvite, inviteConsumed
case sessionRevoked
case redemptionUnavailable
case staleState
case dateOutOfRange
case notProfileHolder, inviteNotResendable
```



App de Organização Familiar · Base funcional 1.4 · Figura 3

O recorte mostra as relações internas deste domínio; as relações entre domínios estão na tabela de relacionamentos.

FamilyTask «class»

Rastreabilidade: UC03, UC05, UC07, UC08, UC11, UC12, UC15, UC17.

```
+ id: UUID
+ familyID: UUID
+ routineID: UUID?
+ title: String
+ weight: EffortWeight
+ date: Date
+ assigneeIDs: Set<UUID>
+ status: TaskStatus
+ completions: [Completion]
+ signals: [KidsSignal]
+ isUnassigned: Bool { get }
+ currentCompletion: Completion? { get }
+ pendingSignal: KidsSignal? { get }
+ edit(title: String, weight: EffortWeight, by actor: Member,
    missions: [Mission]) throws
+ assign(_ ids: Set<UUID>, by actor: Member, missions: [Mission]) throws
+ complete(by actor: Member, takeOver: Bool) throws -> Completion
+ signal(by kid: Member, photo: URL?) throws -> KidsSignal
+ confirmSignal(by admin: Member) throws -> Completion
+ rejectSignal(by admin: Member) throws
+ reopen(by admin: Member, missions: [Mission]) throws
+ canDelete(missions: [Mission]) -> Bool
```

Restrições: Toda tarefa tem título, peso 1-3 e estado único. Só adultos concluem, e apenas nos estados Pendente ou Aguardando confirmação. Só Kids responsáveis sinalizam tarefa pendente. Confirmação, recusa e conclusão manual disputam o mesmo estado; prevalece a primeira válida. A conclusão manual encerra a sinalização pendente; recusa retorna a Pendente. Há no máximo uma conclusão válida e uma sinalização pendente. Histórico preserva as anteriores. Reabrir exige Administrador, missão em andamento e nenhum vínculo com missão concluída, mantém responsáveis e invalida o crédito anterior. Edição e atribuição respeitam proteção das conquistas (RN11, RN12, RN13, RN14, RN16, RN17, RN18, RN19, RN20, RN21, RN22, RN23, RN24, RN25, RN26, RN48, RN54, RN55, RN56, RN57, RN58, RN59).

TaskStatus: String, Codable «enum»

Rastreabilidade: UC07, UC15.

```
case pending
case awaitingConfirmation
case completed
```

EffortWeight: Int, Codable «enum»

Rastreabilidade: UC07.

```
case low = 1
case medium = 2
case high = 3
// ---
+ label: String { get }
```

CompletionSource: String, Codable «enum»

Rastreabilidade: UC03, UC12, UC13, UC17.

```
+ id: UUID
+ taskID: UUID
+ performedBy: UUID
+ creditedAssignees: Set<UUID>
+ creditedWeight: EffortWeight
+ completedAt: Date
+ source: CompletionSource
+ confirmedBy: UUID?
+ undoneAt: Date?
+ undoneBy: UUID?
+ isValid: Bool { get }  // undoneAt == nil
```

Restrições: Responsáveis finais e peso são o registro do crédito daquela conclusão. Cada responsável recebe o peso integral; autor da ação só recebe crédito se for responsável. Confirmação identifica o Administrador; reabertura registra autor e momento, sem apagar o histórico. Progresso usa a data prevista e o estado atual de FamilyTask, não o histórico de créditos (RN25, RN30, RN55, RN56, RN57, RN61).

KidsSignal «struct»

Rastreabilidade: UC03, UC15.

```
+ id: UUID
+ signaledBy: UUID
+ signaledAt: Date
+ photoURL: URL?
+ decision: ValidationDecision?
+ decidedBy: UUID?
+ decidedAt: Date?
```

ValidationDecision: String, Codable «enum»

Rastreabilidade: UC03.

```
case confirmed
case rejected
case closedByManualCompletion
```

Completion «struct»

Rastreabilidade: UC03, UC12, UC13, UC17.

```
+ id: UUID
+ taskID: UUID
+ performedBy: UUID
+ creditedAssignees: Set<UUID>
+ creditedWeight: EffortWeight
+ completedAt: Date
+ scheduledDate: Date  // data prevista (RN61)
+ source: CompletionSource
+ confirmedBy: UUID?
+ undoneAt: Date?  // reabertura; registro preservado (UC17)
// ---
+ isValid: Bool { get }  // undoneAt == nil
```

Routine «class»

Rastreabilidade: UC07, UC08.

```
+ id: UUID
+ familyID: UUID
+ title: String
+ weight: EffortWeight
+ assigneeIDs: Set<UUID>
+ frequency: RoutineFrequency
+ startDate: Date
+ isActive: Bool
+ upcomingDates(until: Date, calendar: Calendar) -> [Date]
+ generateTasks(until: Date, calendar: Calendar) throws -> [FamilyTask]
+ update(title: String, weight: EffortWeight, assignees: Set<UUID>,
    frequency: RoutineFrequency, from: Date, by actor: Member,
    missions: [Mission]) throws
+ end(from: Date, by actor: Member, missions: [Mission]) throws
```

Restrições: Gera ocorrências com IDs e estados próprios, sem duplicar uma ocorrência em retentativa. Alterações afetam somente tarefas futuras abrangidas e preservam dados passados, concluídos e protegidos. Não propagam conclusão, assunção individual ou vínculo com missão para outras ocorrências. until é limite técnico de geração, não prazo da rotina; nenhum horizonte fixo é imposto (RN14, RN16, RN17, RN18, RN48, RN51).

RoutineFrequency: Int, Codable «enum»

Rastreabilidade: UC08.

```
case daily = 1
case every7Days = 7
case every14Days = 14
case every21Days = 21
case every28Days = 28
// sem frequência mensal
```

ChangeScope «enum»

Rastreabilidade: UC07, UC08.

```
case thisOnly
case thisAndFuture
```

TaskService «class»

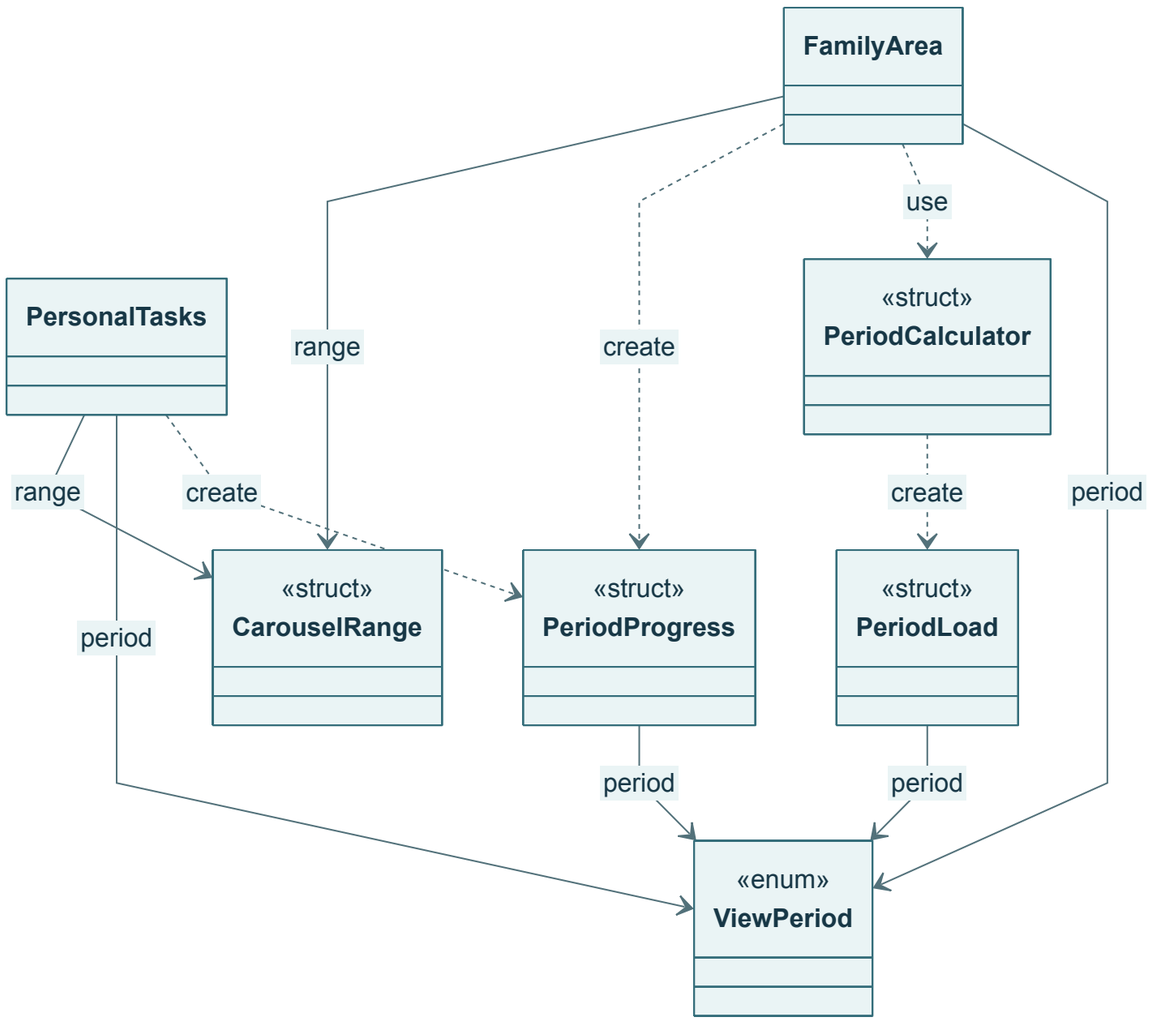
Rastreabilidade: UC03, UC07, UC08, UC12, UC15, UC17.

```
+ create(title: String, weight: EffortWeight, assignees: Set<UUID>,
    date: Date, frequency: RoutineFrequency?, by actor: Member) async throws
    -> [FamilyTask]
+ edit(_ taskID: UUID, title: String, weight: EffortWeight,
    assignees: Set<UUID>, scope: ChangeScope, by actor: Member) async throws
+ delete(_ taskID: UUID, scope: ChangeScope, by actor: Member) async throws
+ complete(_ taskID: UUID, takeOver: Bool, by actor: Member) async throws
+ signal(_ taskID: UUID, photo: URL?, by kid: Member) async throws
+ confirmSignal(_ taskID: UUID, by admin: Member) async throws
+ rejectSignal(_ taskID: UUID, by admin: Member) async throws
+ reopen(_ taskID: UUID, by admin: Member) async throws
+ kidsConfirmationQueue(by admin: Member) async throws -> [FamilyTask]
```

Restrições: Serviço carrega estado e vínculos atuais no backend. Troca de responsáveis só é efetivada com conclusão confirmada; tarefa sem responsável fica atribuída ao adulto que conclui. Conclusão, créditos, atualização das missões e concessão de badges são consistentes; reabertura atualiza créditos/progressos sem alterar conquista preservada. Exclusão bloqueia concluídas e operações que deixem missão vazia; exclusão após reabertura não subtrai crédito outra vez. Alterações de rotina preservam ocorrências protegidas (RN48, RN50, RN53, RN55, RN56, RN57, RN58, RN59, RNF11, RNF25).

-

Área Família, progresso e carga



App de Organização Familiar · Base funcional 1.4 · Figura 4

O recorte mostra as relações internas deste domínio; as relações entre domínios estão na tabela de relacionamentos.

PersonalTasks «class»

Rastreabilidade: UC11, UC15.

```
+ user: Member
+ selectedDate: Date
+ period: ViewPeriod
+ range: CarouselRange
+ init(user: Member, today: Date, calendar: Calendar) throws
+ select(date: Date, period: ViewPeriod) throws
+ tasks() async throws -> [FamilyTask]
+ progress() async throws -> PeriodProgress
+ ownHighlight(week: DateInterval) async throws -> WeeklyRecognition?
```

Restrições: Todos os perfis consultam suas tarefas e progresso, com início no dia atual, Dia/Semana e limites do carrossel. Kids têm duas abas, Minhas tarefas e Missões e conquistas; perfil pelo avatar/cabeçalho. Sinalização com foto abre diretamente a câmera, permite revisar/refazer/descartar e continuar sem foto em caso de cancelamento ou indisponibilidade. Reconhecimento próprio não libera carga familiar (RN27, RN33, RN54, RN61, RN63, RN65; RF55, RF65).

FamilyArea «class»

Rastreabilidade: UC11, UC13, UC18.

```
+ user: Member
+ selectedDate: Date
+ period: ViewPeriod
+ selectedMemberID: UUID?
+ range: CarouselRange
+ init(user: Member, today: Date, calendar: Calendar) throws
+ select(date: Date, period: ViewPeriod) throws
+ select(memberID: UUID?) throws
+ interval: DateInterval { get }
+ availableTaskInterval: DateInterval { get }
+ members() async throws -> [Member]
+ progress() async throws -> PeriodProgress
+ tasks() async throws -> [FamilyTask]
+ load() async throws -> PeriodLoad
+ weeklyReportPDF() async throws -> URL
+ canManageMembers: Bool { get }
+ canValidateKids: Bool { get }
```

Restrições: Entrada restrita a Administradores e Membros, com carga/progresso, lista e PDF diretamente na tela. nil seleciona a família inteira, incluindo tarefas sem responsável. Detalhe herda período, filtra membro e restaura contexto ao voltar; dias disponíveis são indicados nas semanas de borda. Adicionar membro, edição administrativa, convites e validação Kids exigem Administrador. PDF usa a semana da data selecionada mesmo na consulta Dia (RN49, RN60, RN61, RN62).

CarouselRange «struct»

Rastreabilidade: UC11.

```
+ start: Date  // data de criação da família
+ end: Date    // último dia do mês corrente
// ---
+ init(familyCreatedAt: Date, today: Date, calendar: Calendar)
+ contains(_ date: Date) -> Bool
```

ViewPeriod: String, Codable «enum»

Rastreabilidade: UC11, UC13.

```
case day
case week
```

PeriodProgress «struct»

Rastreabilidade: UC11, UC18.

```
+ interval: DateInterval
+ period: ViewPeriod
+ memberID: UUID?  // nil = família
+ completed: Int
+ total: Int
// ---
+ percentage: Double? { get }  // nil = "Sem tarefas"
+ static personal(of m: Member, tasks: [FamilyTask],
      in interval: DateInterval) -> Self
+ static family(tasks: [FamilyTask], in interval: DateInterval) -> Self
  // tarefas contadas pela data prevista (FamilyTask.date)
```

PeriodLoad «struct»

Rastreabilidade: UC13, UC18, UC19.

```
+ interval: DateInterval
+ period: ViewPeriod
+ loadByMember: [UUID: Int]
+ completedCountByMember: [UUID: Int]
+ distinctCompletedTaskIDs: Set<UUID>
+ total: Int { get }
+ distinctCompletedCount: Int { get }
+ share(of id: UUID) -> Double?
+ completedCount(of id: UUID) -> Int
```

Restrições: Conta apenas conclusões válidas pela data de conclusão. Cada tarefa compartilhada conta uma vez por responsável e uma vez no total familiar distinto. Carga total soma créditos individuais; participação é percentual e fica sem valor quando a carga total é zero (RN30, RN31, RN61, RN62).

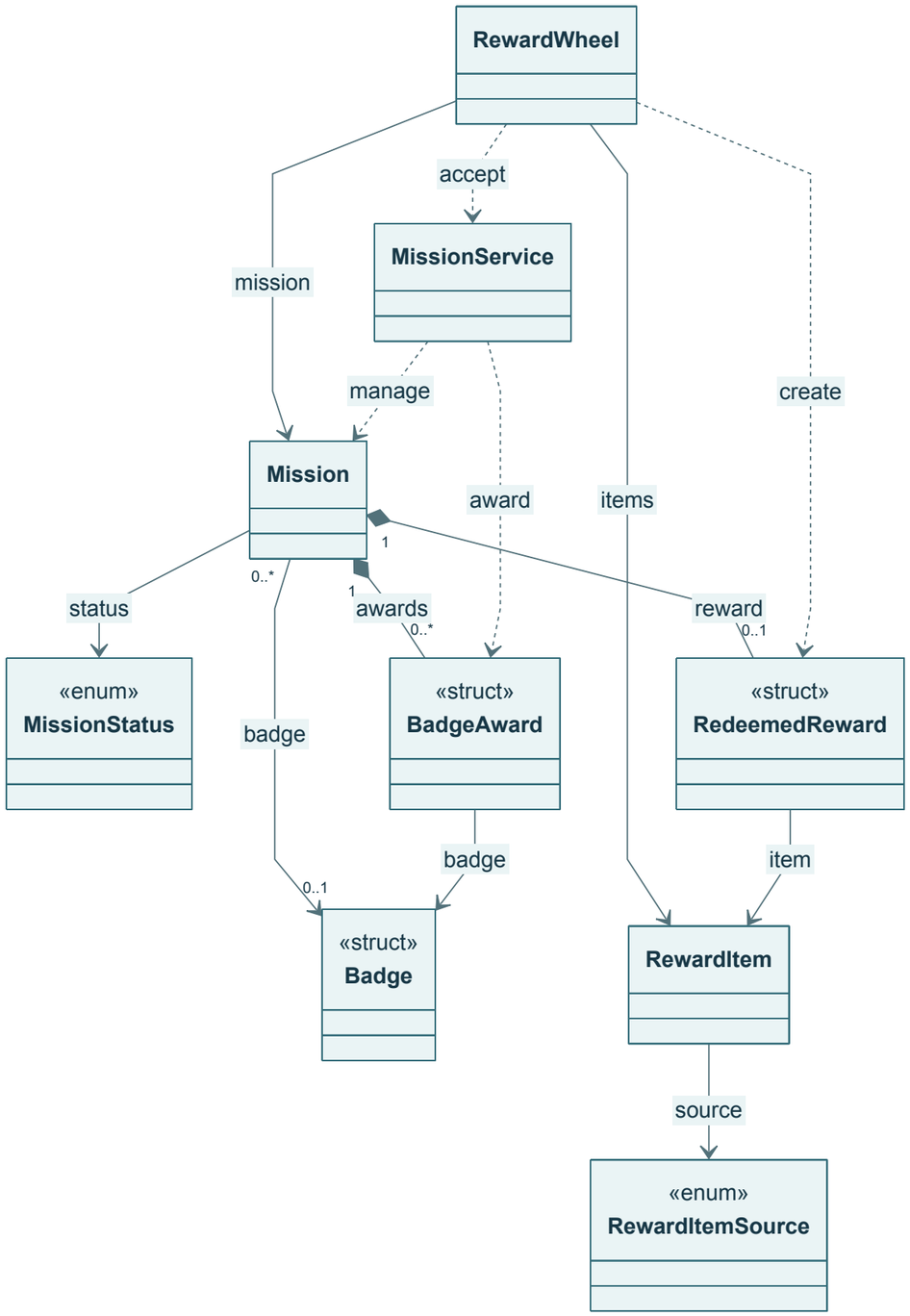
PeriodCalculator «struct»

Rastreabilidade: UC11, UC13, UC18, UC19.

```
+ calendar: Calendar  // mesmo fuso de referência da família
+ interval(_ period: ViewPeriod, containing date: Date) -> DateInterval
+ availableTaskInterval(_ interval: DateInterval, range: CarouselRange) -> DateInterval?
+ load(_ period: ViewPeriod, containing date: Date,
    completions: [Completion]) -> PeriodLoad
```

Restrições: Semana de segunda a domingo no fuso comum, sem depender do dispositivo. Progresso semanal usa totais, nunca média dos percentuais diários. Carga usa completedAt de conclusões válidas; progresso usa data prevista/estado atual. Fuso concreto e horário operacional seguem a definição pendente da RN28, sem configuração pelo usuário (RN28, RN61).

Missões e recompensas



App de Organização Familiar · Base funcional 1.4 · Figura 5

O recorte mostra as relações internas deste domínio; as relações entre domínios estão na tabela de relacionamentos.

Mission «class»

Rastreabilidade: UC05, UC14, UC16, UC17.

```
+ id: UUID
+ familyID: UUID
+ title: String
+ createdBy: UUID
+ taskIDs: Set<UUID>
+ badge: Badge?
+ status: MissionStatus
+ completedAt: Date?
+ reward: RedeemedReward?
+ awards: [BadgeAward]
+ isRedemptionAvailable: Bool { get }
+ progress(tasks: [FamilyTask]) -> Double
+ edit(title: String, badge: Badge?, by admin: Member) throws
+ link(_ task: FamilyTask, by admin: Member) throws
+ unlink(_ taskID: UUID, replacement: FamilyTask?, by admin: Member) throws
+ recordReward(_ reward: RedeemedReward, by admin: Member) throws
```

Restrições: Ao menos uma tarefa individual da mesma família; cada tarefa conta uma vez no percentual. Missão não tem prazo. Dados e vínculos de missão concluída são imutáveis, exceto registro único da recompensa aceita. Remover a última tarefa exige substituição na mesma operação. Estado e badges são atualizados pelo serviço ao concluir tarefas, sem atribuição manual pelo usuário (RN35, RN36, RN37, RN38, RN39, RN40, RN50, RN51, RN52, RN53, RN58, RN64).

MissionService «class»

Rastreabilidade: UC03, UC05, UC10, UC12, UC14, UC16, UC17.

```
+ create(title: String, taskIDs: Set<UUID>, badge: Badge?,
    by admin: Member) async throws -> Mission
+ edit(_ missionID: UUID, title: String, taskIDs: Set<UUID>,
    badge: Badge?, by admin: Member) async throws
+ updateAfterTaskChange(_ taskID: UUID) async throws
+ acceptReward(_ item: RewardItem, for missionID: UUID,
    by admin: Member) async throws -> RedeemedReward
+ list(by actor: Member) async throws -> [Mission]
+ badges(of memberID: UUID, by actor: Member) async throws -> [BadgeAward]
```

Restrições: Criar ou editar missão também recalcula seu estado: se todas as tarefas vinculadas já estiverem concluídas, conclui e concede o selo nessa operação. Atualização participa da operação consistente de tarefa: todas concluídas encerram a missão, concedem selo coletivo quando houver e liberam resgate. Concessão é única por missão/membro, independente de responsáveis e resgate; adota perfis ativos conforme premissa RN64. Aceitação valida item, permissão e disponibilidade, grava recompensa e encerra resgate atomicamente. Todos os perfis consultam missões/conquistas da família (RN38, RN53, RN64, RNF23, RNF25).

MissionStatus: String, Codable «enum»

Rastreabilidade: UC05, UC14.

```
case inProgress
case completed
```

Badge «struct»

Rastreabilidade: UC05, UC10, UC14.

```
+ id: UUID
+ name: String
+ icon: String  // SF Symbol
```

BadgeAward «struct»

Rastreabilidade: UC05, UC10, UC14.

```
+ memberID: UUID
+ missionID: UUID
+ badgeID: UUID
+ awardedAt: Date
```

Restrições: Identidade composta por membro e missão. Não duplica concessão; missão sem selo não gera badge. Remoção de membro não apaga conquista histórica; tocar no badge abre a missão e voltar retorna ao perfil (RN64, UC10).

RewardWheel «class»

Rastreabilidade: UC16.

```
+ mission: Mission
+ items: [RewardItem]
+ currentResult: RewardItem?
+ init(mission: Mission, items: [RewardItem], by admin: Member) throws
+ spin(by admin: Member) throws -> RewardItem
+ rejectResult(by admin: Member) throws
+ accept(by admin: Member) async throws -> RedeemedReward
+ close()
```

Restrições: Só Administradores abrem, sorteiam ou aceitam, em missão concluída e sem resgate. Itens incluem catálogo e personalizados da família. Recusar permite novo sorteio; fechar sem aceitar preserva conquista e disponibilidade. Aceitar usa MissionService.acceptReward, revalidando estado concorrente; resultado em memória não garante resgate (RN42, RN43, RN52, RN53).

RedeemedReward «struct»

Rastreabilidade: UC14, UC16.

```
+ itemID: UUID
+ title: String
+ acceptedBy: UUID
+ acceptedAt: Date
```

RewardItem «class»

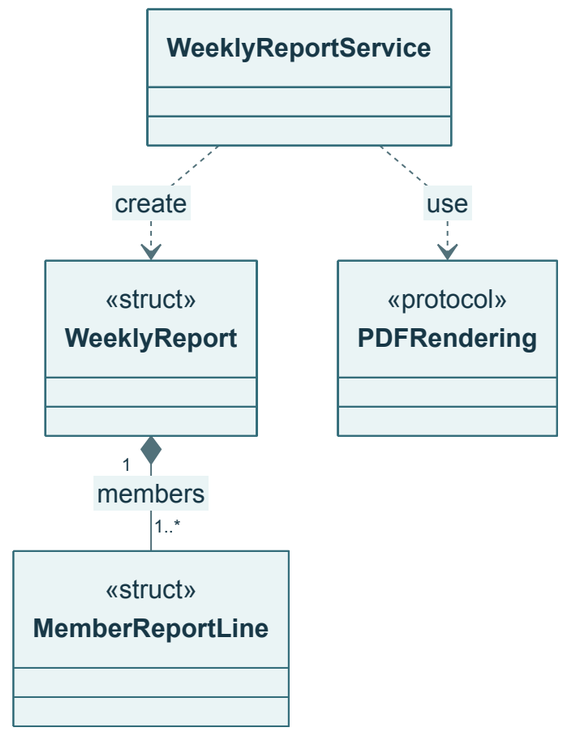
Rastreabilidade: UC06, UC16.

```
+ id: UUID
+ familyID: UUID?  // nil = predefinida
+ title: String
+ details: String?
+ source: RewardItemSource
+ createdBy: UUID?
// ---
+ static predefined: [RewardItem]
+ static custom(title: String, by admin: Member) throws -> Self
```

RewardItemSource: String, Codable «enum»

Rastreabilidade: UC06.

```
case predefined
case custom
```



App de Organização Familiar · Base funcional 1.4 · Figura 6

O recorte mostra as relações internas deste domínio; as relações entre domínios estão na tabela de relacionamentos.

WeeklyReport «struct»

Rastreabilidade: UC18.

```
+ id: UUID
+ familyID: UUID
+ familyName: String
+ week: DateInterval  // cabeçalho segunda a domingo
+ generatedAt: Date
+ generatedBy: UUID
+ isPartial: Bool
+ dataAvailableFrom: Date
+ dataAvailableThrough: Date
+ load: PeriodLoad
+ familyProgress: PeriodProgress
+ members: [MemberReportLine]
+ distinctCompletedCount: Int
+ metricExplanation: String
```

Restrições: Semana atual é parcial até a geração; semana de criação informa ausência anterior. Progresso considera tarefas previstas na semana e estado atual, inclusive tarefas futuras dentro dessa semana. Carga e realizadas consideram conclusões válidas até a geração. Total familiar realizado conta tarefas distintas; contagens individuais de tarefa compartilhada não devem ser somadas como total familiar. Sem dados, zeros e “Sem tarefas”. Não inclui fotos de validação, credenciais ou configuração privada (RN62).

WeeklyReportService «class»

Rastreabilidade: UC18.

```
- calculator: PeriodCalculator
- renderer: PDFRendering
+ build(weekOf date: Date, familyID: UUID, by actor: Member) async throws -> WeeklyReport
+ exportPDF(_ report: WeeklyReport, by actor: Member) async throws -> URL
```

Restrições: Autoriza adulto da família e obtém leitura consistente de tarefas/conclusões no backend. Aceita somente semana atual ou passada que intercepte a existência da família; bloqueia semana futura. isPartial, limites de dados e geração são calculados nessa leitura. Gráficos e tabela usam os mesmos valores. Falha não disponibiliza arquivo incompleto; seleção é preservada para retentar. PDF exportado não muda com alterações posteriores; visualizar, salvar e compartilhar dependem da pessoa (RN62, RNF26, RNF27, UC18).

MemberReportLine «struct»

Rastreabilidade: UC18.

```
+ memberID: UUID
+ displayName: String
+ load: Int
+ loadShare: Double?
+ progress: PeriodProgress
+ completedCount: Int
```

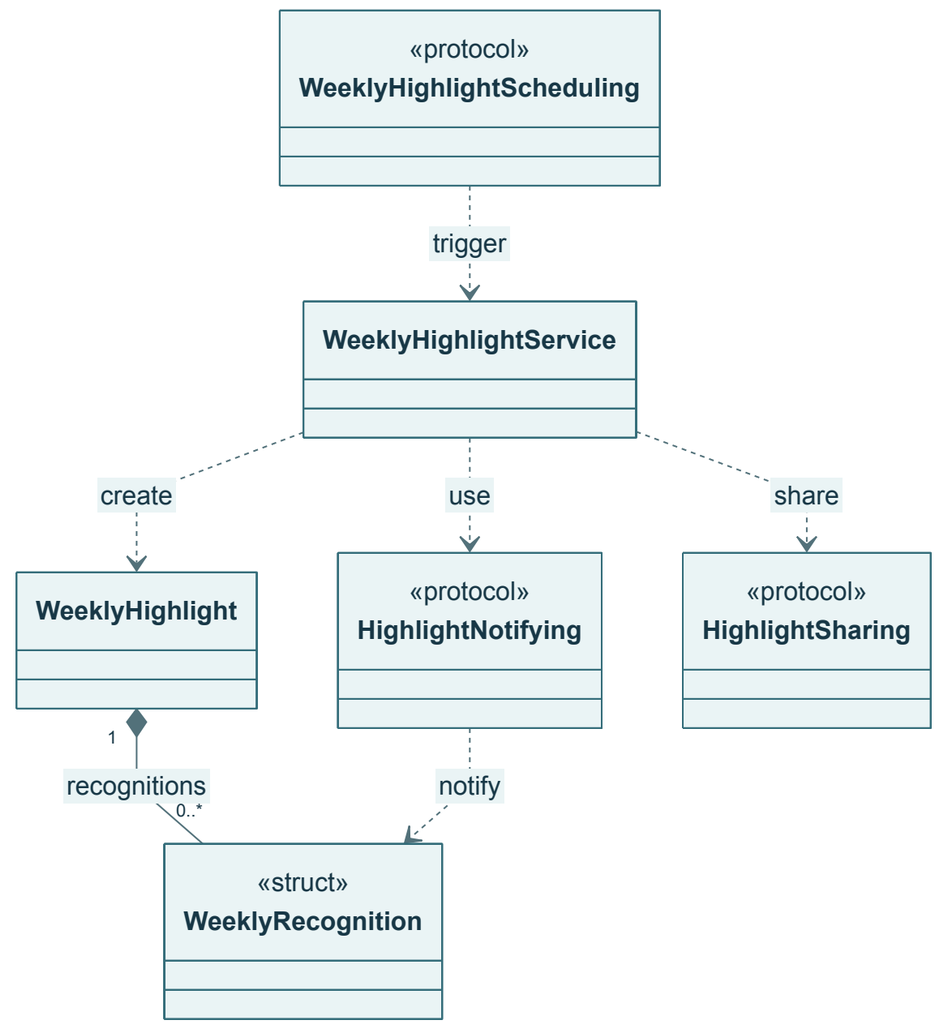
PDFRendering «protocol»

Rastreabilidade: UC18.

```
+ render(_ report: WeeklyReport) async throws -> Data
```

Restrições: Renderiza identificação de família/período/geração, indicação de parcialidade, gráficos de carga por membro e progresso total e tabela de peso creditado/quantidade realizada por membro. Inclui explicação de métricas/crédito compartilhado e ausência de dados; texto complementa cores. Sem conteúdo privado fora da projeção WeeklyReport (RN62, RNF05, RNF26, RNF27).

Destaque semanal



App de Organização Familiar · Base funcional 1.4 · Figura 7

O recorte mostra as relações internas deste domínio; as relações entre domínios estão na tabela de relacionamentos.

WeeklyHighlight «class»

Rastreabilidade: UC19.

```
+ id: UUID
+ familyID: UUID
+ week: DateInterval
+ startedAt: Date
+ processedAt: Date?
+ recognitions: [WeeklyRecognition]
+ isProcessed: Bool { get }  // processedAt != nil
+ hasWinner: Bool { get }
+ isTie: Bool { get }
```

Restrições: Único por família/semana. Resultado fica visível somente após processamento completo. processedAt preenchido com lista vazia representa ausência de vencedor; nil representa processamento incompleto recuperável. Empate produz um reconhecimento por pessoa, todos com a maior pontuação positiva. Revisão após reabertura tardia permanece a decisão pendente de RN65.

WeeklyRecognition «struct»

Rastreabilidade: UC11, UC19.

```
+ highlightID: UUID
+ memberID: UUID
+ week: DateInterval
+ points: Int
+ completedCount: Int
+ notifiedAt: Date?
+ notificationKey: String { get }
```

Restrições: Identidade e chave de notificação por apuração/membro. Guarda pontos e quantidade realizada da semana encerrada; notifiedAt é individual, registrado após entrega confirmada. Consulta exige sessão válida do próprio reconhecido, inclusive Kids; perfil removido não acessa. Não expõe carga nem dados dos demais membros (RN33, RN65, RNF26, RNF27).

WeeklyHighlightService «class»

Rastreabilidade: UC19.

```
- calculator: PeriodCalculator
- notifier: HighlightNotifying
- sharing: HighlightSharing
+ computePreviousWeek(familyID: UUID, referenceDate: Date) async throws -> WeeklyHighlight
+ notifyPending(_ highlightID: UUID) async throws
+ ownRecognition(week: DateInterval, by actor: Member) async throws -> WeeklyRecognition?
+ prepareShare(_ recognition: WeeklyRecognition, by holder: Member) throws -> String
+ share(editedMessage: String, recognition: WeeklyRecognition,
    by holder: Member) async throws
```

Restrições: Apura semana anterior encerrada usando conclusões válidas e a mesma métrica da carga, sem segunda moeda ou penalidades. Considera perfis ativos inclusive Kids e registra ausência de vencedor sem pontos positivos, conforme premissas RN65. Retentativa recupera processamento incompleto; resultado já finalizado é reutilizado, sem duplicar reconhecimentos. Notifica cada destinatário pendente com chave estável, sem publicar resultados parciais. Consulta/compartilhamento revalidam titularidade. Mensagem editável contém apenas o reconhecimento próprio e convite à atividade em família; envio é opcional (RN65, RNF26, RNF27).

HighlightNotifying «protocol»

Rastreabilidade: UC19.

```
+ notify(_ recognition: WeeklyRecognition, idempotencyKey: String) async throws -> Date
```

Restrições: Entrega no máximo uma notificação por chave apuração/membro e retorna confirmação de entrega. Repetição da mesma chave recupera a confirmação anterior, inclusive após falha entre entrega e persistência local. Falha ou falta de permissão externa não remove o resultado acessível no app (RN65, RNF26).

WeeklyHighlightScheduling «protocol»

Rastreabilidade: UC19.

```
+ schedulePreviousWeekComputation(familyID: UUID, calendar: Calendar) async throws
+ retryIncompleteComputation(familyID: UUID, week: DateInterval) async throws
```

Restrições: Aciona WeeklyHighlightService na segunda-feira para a semana encerrada, independentemente de app aberto ou sessão humana. Executa em contexto autorizado do sistema para a família; fuso e horário operacional serão os definidos pela base funcional (RN28, RN65, UC19).

HighlightSharing «protocol»

Rastreabilidade: UC19.

> + isWhatsAppAvailable: Bool { get }
> + openWhatsApp(editableMessage: String) async throws
> + copyMessage(_ message: String) throws

Restrições: Acionado somente pela pessoa reconhecida. Ela escolhe destinatário, revisa e confirma no WhatsApp; abrir aplicativo externo não registra envio. Se indisponível, oferece copiar; cancelamento ou falha mantém reconhecimento. Texto convida: “Mande para sua família e sugira algo para vocês fazerem juntos.” (RN65, UC19).

Relacionamentos

Origem	Relação	Destino	Rótulo	Multiplicidade (origem → destino)
Family	composição ◆	Member	members	1 → 1..*
Member	associação →	MemberRole	role	– → 1
Invite	associação →	Member	for	0..* → 1
Invite	associação →	InviteStatus	status	– → 1
Session	associação →	Member	of	0..* → 1
AccessService	dependência ⇢	AppleAuthenticating	«use»	– → –
AccessService	dependência ⇢	Invite	«create»	– → –
AccessService	dependência ⇢	AccessControl	«use»	– → –
AccessControl	dependência ⇢	Action	«use»	– → –
Family	composição ◆	FamilyTask	tasks	1 → 0..*
FamilyTask	associação →	Member	assignees	0..* → 0..*
FamilyTask	associação →	TaskStatus	status	– → 1
FamilyTask	composição ◆	Completion	completions	1 → 0..*
FamilyTask	composição ◆	KidsSignal	signals	1 → 0..*
Completion	associação →	CompletionSource	source	– → 1
KidsSignal	associação →	ValidationDecision	decision	– → 0..1
Routine	associação →	FamilyTask	generates	0..1 → 0..*
Routine	associação →	RoutineFrequency	frequency	– → 1
TaskService	dependência ⇢	ChangeScope	«use»	– → –
FamilyArea	associação →	CarouselRange	range	– → 1
FamilyArea	dependência ⇢	PeriodProgress	«create»	– → –
FamilyArea	dependência ⇢	PeriodCalculator	«use»	– → –
PeriodCalculator	dependência ⇢	PeriodLoad	«create»	– → –

Origem	Relação	Destino	Rótulo	Multiplicidade (origem → destino)
Mission	associação →	FamilyTask	links	0..* → 1..*
Mission	associação →	MissionStatus	status	– → 1
Mission	composição ◆	RedeemedReward	reward	1 → 0..1
Mission	associação →	Badge	badge	0..* → 0..1
RewardWheel	associação →	Mission	mission	– → 1
RewardWheel	associação →	RewardItem	items	– → 1..*
RewardWheel	dependência ⇢	RedeemedReward	«create»	– → –
RedeemedReward	associação →	RewardItem	item	– → 1
RewardItem	associação →	RewardItemSource	source	– → 1
FamilyArea	associação →	ViewPeriod	period	– → 1
Mission	composição ◆	BadgeAward	awards	1 → 0..*
BadgeAward	associação →	Badge	badge	– → 1
WeeklyReport	composição ◆	MemberReportLine	members	1 → 1..*
WeeklyReportService	dependência ⇢	WeeklyReport	«create»	– → –
WeeklyReportService	dependência ⇢	PDFRendering	«use»	– → –
WeeklyReportService	dependência ⇢	PeriodCalculator	«use»	– → –
FamilyArea	dependência ⇢	WeeklyReportService	«use»	– → –
WeeklyHighlightService	dependência ⇢	WeeklyHighlight	«create»	– → –
WeeklyHighlightService	dependência ⇢	HighlightNotifying	«use»	– → –
MemberService	dependência ⇢	Member	manage	– → –
MemberService	dependência ⇢	AccessService	revoke	– → –
MemberService	dependência ⇢	TaskService	releaseFutureTasks	– → –
AccessService	dependência ⇢	InviteCredentials	provide	– → –
AccessControl	dependência ⇢	Session	validate	– → –
TaskService	dependência ⇢	MissionService	update	– → –

Origem	Relação	Destino	Rótulo	Multiplicidade (origem → destino)
FamilyTask	associação →	EffortWeight	weight	– → 1
PersonalTasks	associação →	CarouselRange	range	– → 1
PersonalTasks	associação →	ViewPeriod	period	– → 1
PersonalTasks	dependência ⇢	PeriodProgress	create	– → –
PersonalTasks	dependência ⇢	WeeklyHighlightService	ownRecognition	– → –
PeriodLoad	associação →	ViewPeriod	period	– → 1
PeriodProgress	associação →	ViewPeriod	period	– → 1
MissionService	dependência ⇢	Mission	manage	– → –
MissionService	dependência ⇢	BadgeAward	award	– → –
RewardWheel	dependência ⇢	MissionService	accept	– → –
BadgeAward	associação →	Member	holder	0..* → 1
WeeklyReport	associação →	PeriodLoad	load	– → 1
WeeklyReport	associação →	PeriodProgress	progress	– → 1
WeeklyHighlight	composição ◆	WeeklyRecognition	recognitions	1 → 0..*
WeeklyRecognition	associação →	Member	holder	0..* → 1
WeeklyHighlightScheduling	dependência ⇢	WeeklyHighlightService	trigger	– → –
WeeklyHighlightService	dependência ⇢	PeriodCalculator	calculate	– → –
WeeklyHighlightService	dependência ⇢	HighlightSharing	share	– → –
HighlightNotifying	dependência ⇢	WeeklyRecognition	notify	– → –

Regras centrais do modelo

Tipos	Invariantes de comportamento	Base
FamilyTask, KidsSignal, Completion	Uma tarefa tem estado único para todos os responsáveis. Sinalização Kids, com foto opcional, não conclui parcialmente. A primeira decisão válida prevalece; conclusão manual encerra a validação.	RN23, RN24, RN54, RN55
FamilyTask, Completion	Cada responsável final recebe o peso integral. Quem executa a ação pode não receber crédito; assumir substitui todos os responsáveis, e concluir sem responsável atribui a tarefa ao adulto que conclui.	RN30, RN48, RN56
PeriodProgress, PeriodLoad, PeriodCalculator	Progresso conta tarefas distintas pela data prevista; carga soma créditos válidos pela data de conclusão, nos períodos Dia/Semana. Total de carga é a soma das cargas individuais; ausência de dados não permite divisão por zero.	RN28, RN30, RN31, RN60, RN61
FamilyArea, CarouselRange	Carrossel vai da criação da família ao fim do mês corrente; contexto é preservado no retorno. Kids consultam o próprio progresso, mas não acessam a área Família ou a carga familiar.	RN33, RN49, RN60, RN61
Mission, FamilyTask, Completion	Missões não expiram e têm ao menos uma tarefa individual vinculada. Missão concluída preserva dados e vínculos. Reabrir exige Administrador, missão em andamento e ausência de missão concluída vinculada; desfaz o crédito anterior, sem apagar o histórico ou descontá-lo outra vez na exclusão posterior.	RN36, RN50, RN51, RN57, RN58, RN59
RewardWheel, RedeemedReward, BadgeAward	Recompensa só é vinculada após sorteio e aceitação, em resgate único. Badge de missão é coletivo, sem duplicação por membro/missão e independente do resgate.	RN52, RN53, RN64

Tipos	Invariantes de comportamento	Base
Invite, Session, Member	Link e senha permitem um ingresso. Sessão usa refresh token sem expiração temporal, com revogação; recuperação preserva o mesmo perfil ativo. Apelido é visível à família e editável só pelo titular.	RN46, RN47, RN63
WeeklyReport, WeeklyHighlight	PDF é semanal e exclusivo de adultos. Apuração reconhece todos os empatados em primeiro; envio externo depende da pessoa. PDF identifica parcialidade; apuração registra processamento recuperável e notificação por pessoa.	RN62, RN65

Correspondência com o DER

O quadro relaciona conceitos com o modelo de dados. Classes descrevem comportamento e projeções; não exigem uma tabela para cada tipo. O DER tem versão própria e não substitui os contratos funcionais abaixo.

Tipos	Entidades ou origem dos dados	Correspondência
Family, Member, MemberRole	FAMILIA, MEMBRO	Família, perfil, papel e identificação Apple restrita ao Criador em Member.
Invite, InviteStatus, Session	CONVITE, SESSAO	Vínculos, credenciais protegidas, consumo e revogação. InviteCredentials é transitório.
Routine, RoutineFrequency	TAREFA, RECORRENCIA, TAREFA_RESPONSAVEL	Configuração recorrente e responsáveis padrão; ocorrências passadas preservam seus dados. Versões de recorrência são um mecanismo de persistência do DER.
FamilyTask, TaskStatus, EffortWeight	OCORRENCIA_TAREFA, OCORRENCIA_RESPONSAVEL	Tarefa individual prevista e responsáveis. Tarefa única também tem TAREFA no DER, sem Routine no modelo de domínio.

Tipos	Entidades ou origem dos dados	Correspondência
Completion, KidsSignal, CompletionSource, ValidationDecision	Estado da ocorrência e OCORRENCIA_EVENTO	Conclusão, responsáveis/peso creditados, sinalização, decisão e reabertura. O mapeamento de persistência deve conservar esses dados do contrato; o DER não discrimina todos os campos do histórico.
Mission, MissionStatus	MISSAO, MISSAO_OCORRENCIA	Missão e vínculos individuais; estado decorrente das tarefas.
Badge, BadgeAward	SELO, MEMBRO_BADGE	Selo opcional por associação e concessão identificada por missão/membro, sem propriedade exclusiva do selo.
RewardItem, RewardItemSource, RedeemedReward	RECOMPENSA, aceitação em MISSAO	Item elegível e registro da aceitação única. Título/autor da aceitação são dados do contrato cuja persistência deve preservar a conquista.
WeeklyHighlight, WeeklyRecognition	APURACAO_SEMANAL, DESTAQUE_SEMANAL	Processamento por família/semana, com resultado e notificação por reconhecido.
PeriodProgress, PeriodLoad, WeeklyReport, MemberReportLine	Tarefas, conclusões e membros	Cálculos derivados e projeção consistente para PDF, sem duplicação de créditos.
Coordenadores, serviços, protocolos e demais tipos de apoio	Sem tabela própria	Navegação, calendário, autorização, orquestração e integrações.

Definições pendentes na base funcional

O modelo mantém somente as indefinições e premissas registradas na documentação funcional vigente; não estabelece respostas adicionais.

Referência	Definição vigente
RN28, RN65	Mesmo fuso da família em consultas, PDF e apuração; fuso concreto e horário operacional ainda pendentes, sem configuração de fuso pelo usuário.
RN64	Conceder badge aos perfis ativos na conclusão é premissa; concessão retroativa a quem entra depois permanece pendente.

Referência	Definição vigente
RN65	Elegibilidade de todos os perfis ativos, inclusive Kids, e ausência de vencedor sem pontos positivos são premissas. Empates reconhecem todos os primeiros colocados.
RN65	Política de revisão do destaque após reabertura tardia permanece pendente. Carga e novos PDFs refletem as conclusões válidas.